

设有数据结构 (D, R) ，其中 $D = \{1, 2, 3, 4, 5, 6\}$ ， $R = \{(1, 2), (2, 3), (2, 4), (3, 4), (3, 5), (3, 6), (4, 5), (4, 6)\}$ 。试画出其逻辑结构图并指出属于何种结构。

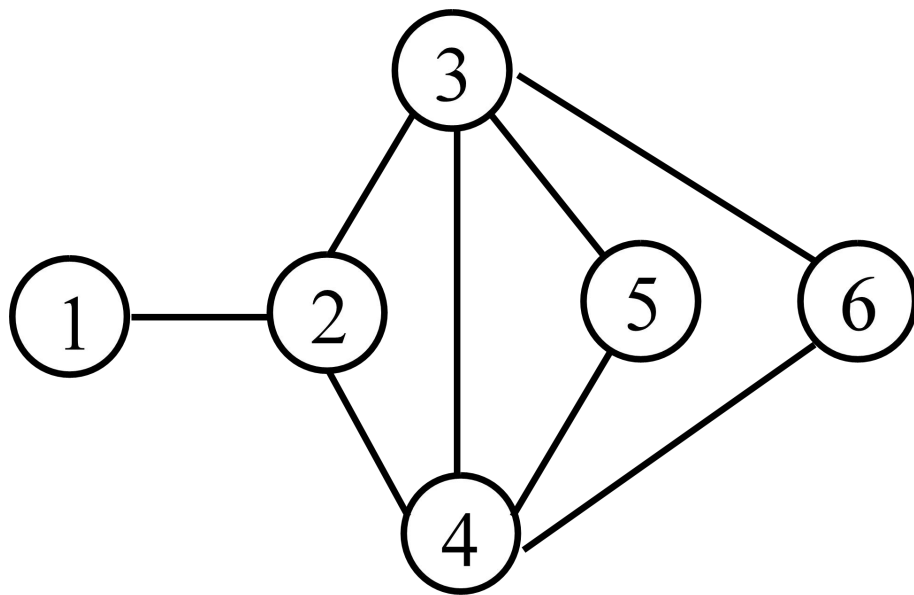
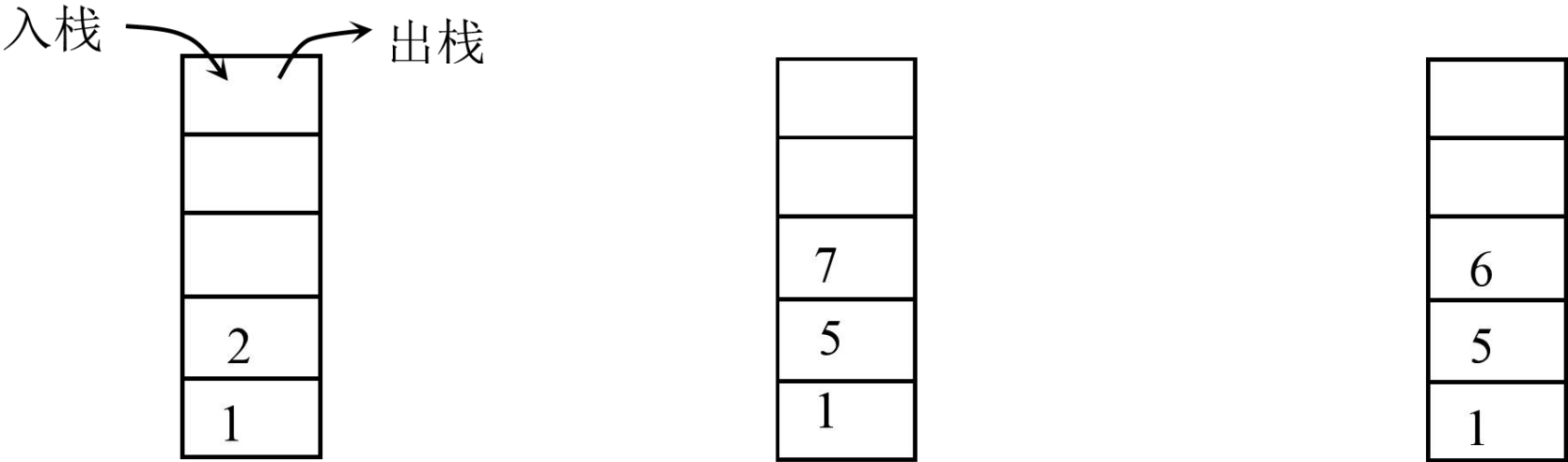


图 1-3 逻辑结构图

在操作序列push(1)、push(2)、pop、push(5)、push(7)、pop、push(6)之后，栈顶元素和栈底元素分别是什么？（push(k)表示整数k入栈，pop表示栈顶元素出栈。

栈顶元素为6，栈底元素为1。其执行过程如图3-6所示。



(a) push(1)，push(2) (b) pop，push(5)，push(7) (c) pop，push(6)

图 3-6 栈的执行过程示意图

在操作序列EnQueue(1)、 EnQueue(3)、 DeQueue、 EnQueue(5)、 EnQueue(7)、 DeQueue、 EnQueue(9)之后，队头元素和队尾元素分别是什么？（EnQueue(k)表示整数k入队， DeQueue表示队头元素出队）。

队头元素为5，队尾元素为9。其执行过程如图3-7所示。

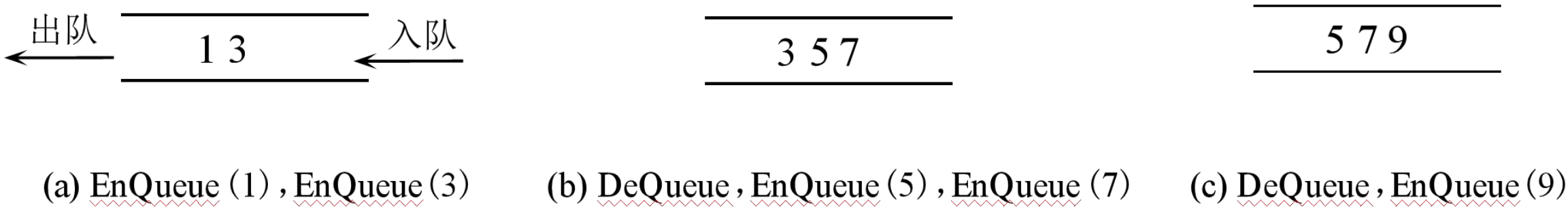


图 3-7 队列的执行过程示意图

已知二叉树的中序和后序序列分别为**CBEDAFIGH**和**CEDBIFHGA**，试构造该二叉树。

二叉树的构造过程如图5-10 所示。

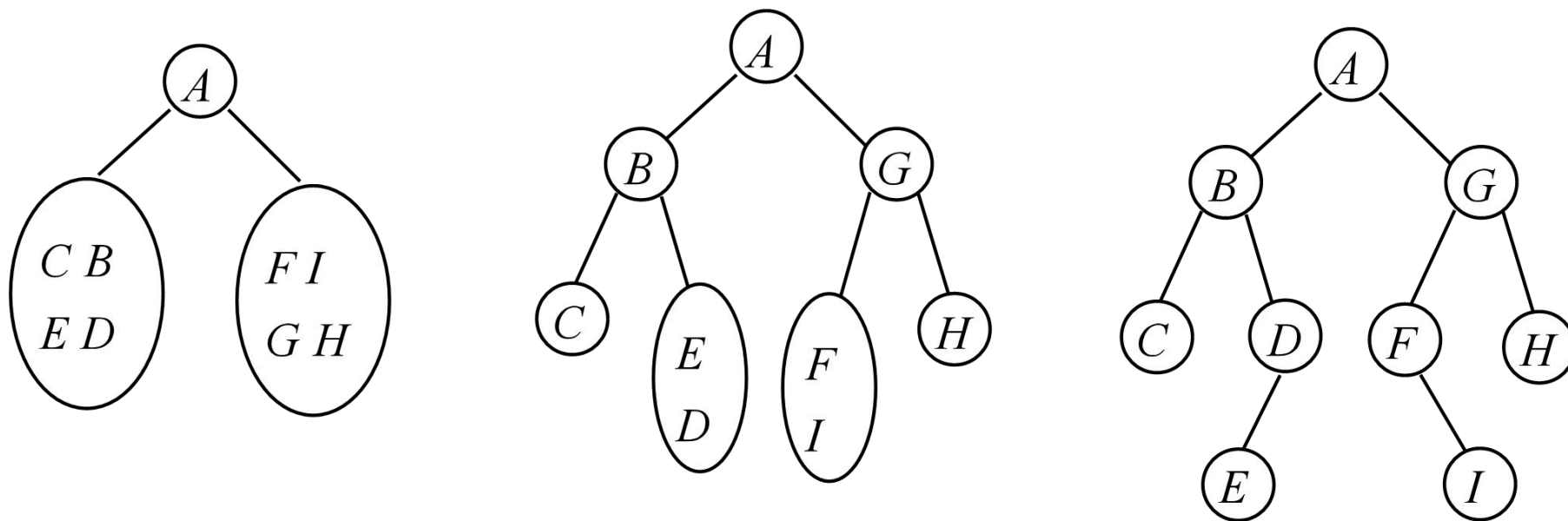
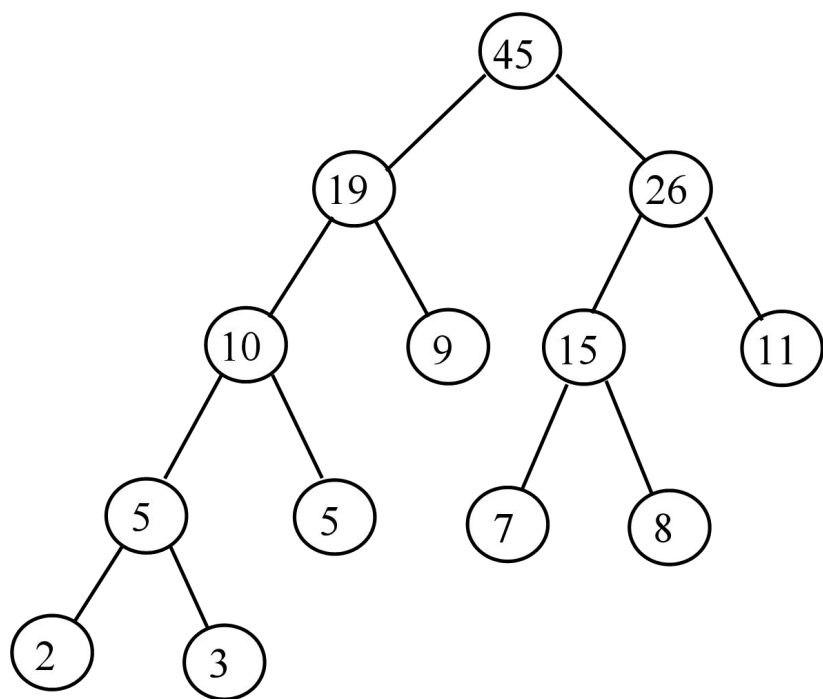


图 5-10 构造二叉树的过程

对给定的一组权值 $W = (5, 2, 9, 11, 8, 3, 7)$ ，试构造相应的哈夫曼树，并计算它的带权路径长度。

构造的哈夫曼树如图5-11所示。

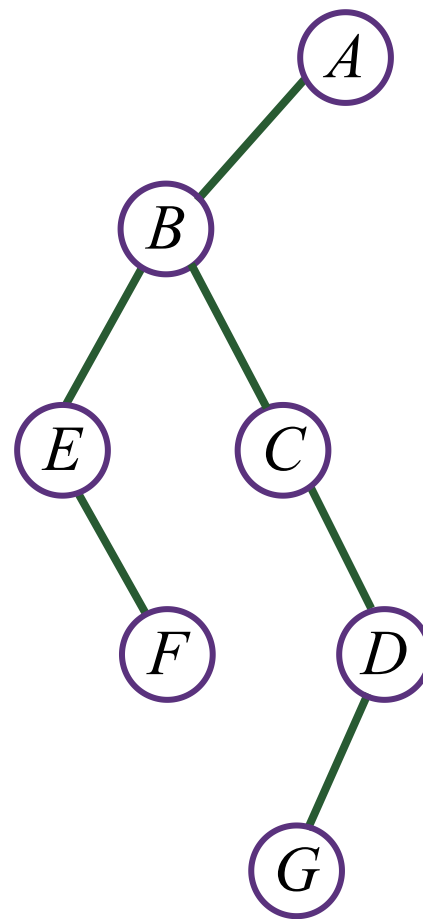
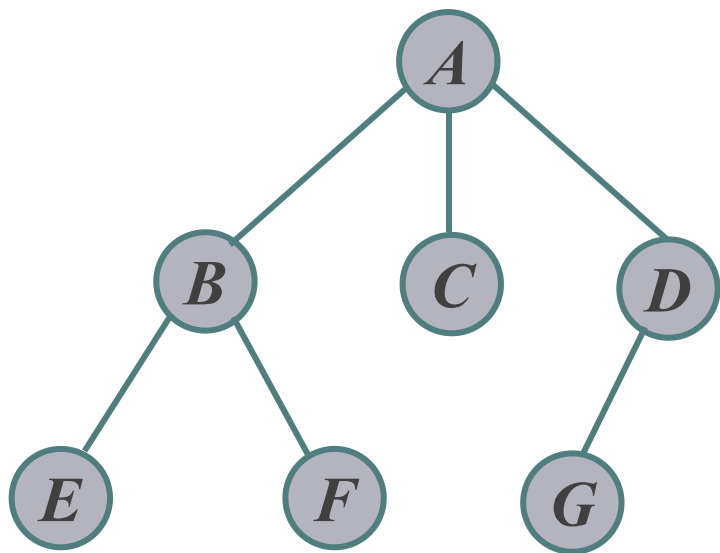


树的带权路径长度为：

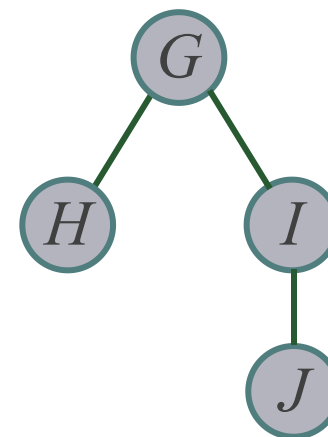
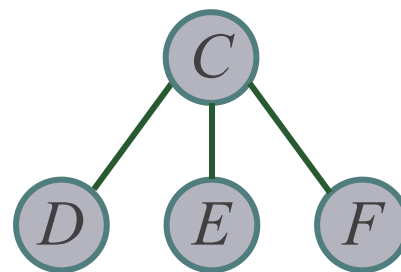
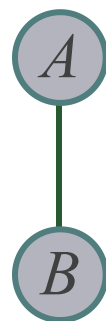
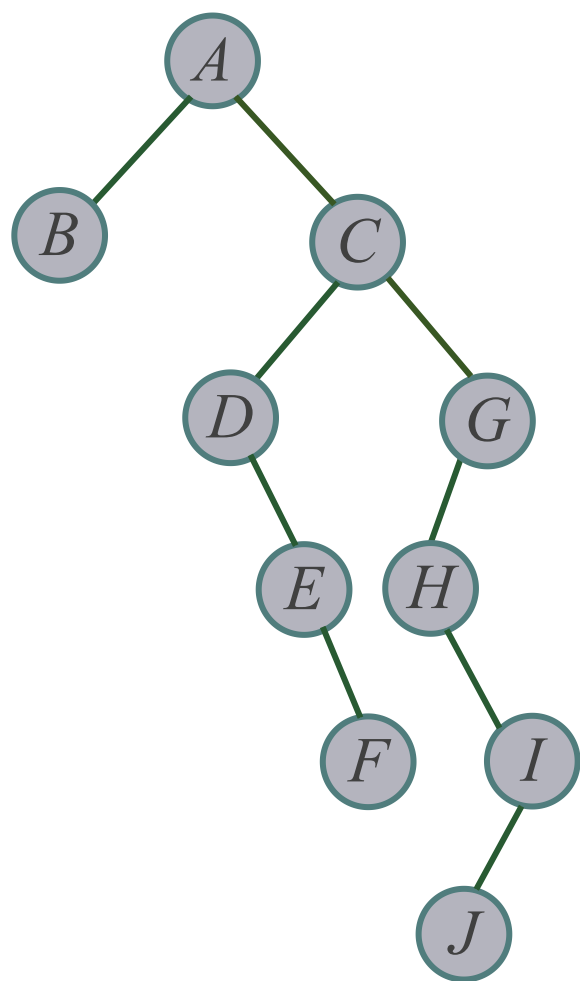
$$2 \times 4 + 3 \times 4 + 5 \times 3 + 7 \times 3 + 8 \times 3 + 9 \times 2 + 11 \times 2 = 120$$

图 5-11 构造的哈夫曼树及带权路径长度

树转换为二叉树



二叉树转换为树或森林



已知一个连通图如图6-8所示，试给出图的邻接矩阵和邻接表存储示意图，若从顶点v1出发对该图进行遍历，分别给出一个按深度优先遍历和广度优先遍历的顶点序列。

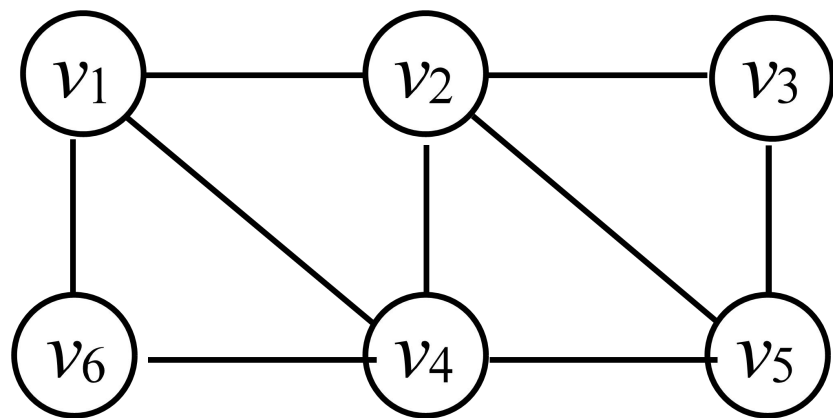


图 6-8 第 (4) 题图

邻接矩阵表示如下：

$$\begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 1 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

深度优先遍历序列为：v1 v2 v3 v5 v4 v6

广度优先遍历序列为：v1 v2 v4 v6 v3 v5

邻接表表示如下：

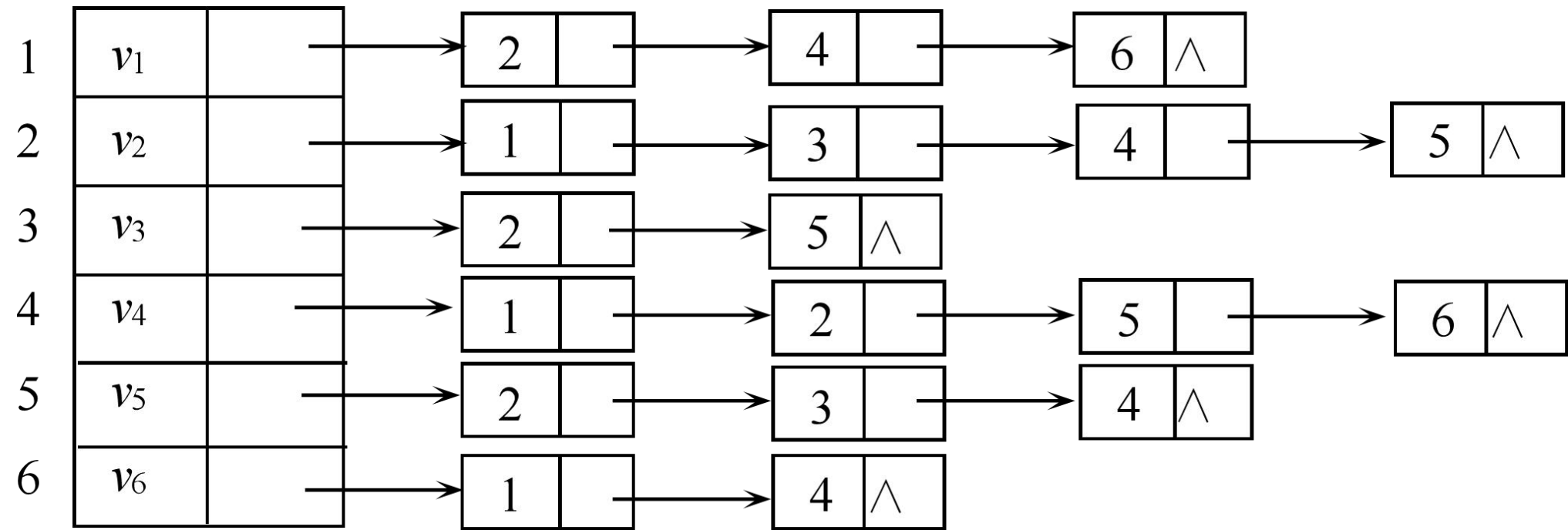


图6-9所示是一个无向带权图，请分别按Prim算法和Kruskal算法求最小生成树。

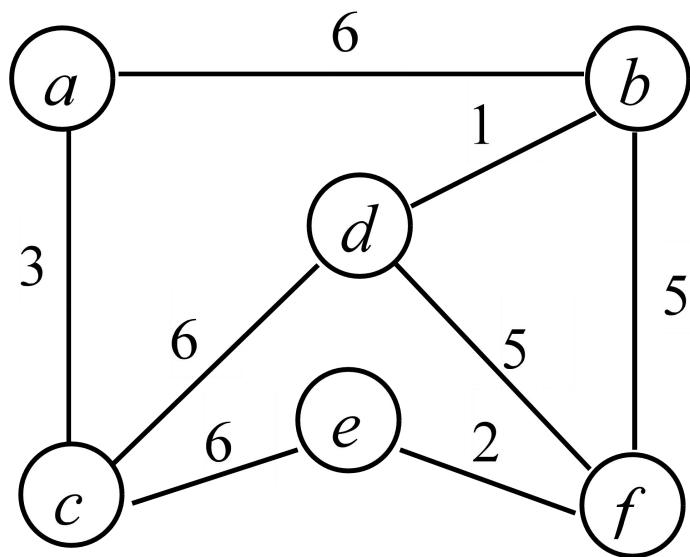
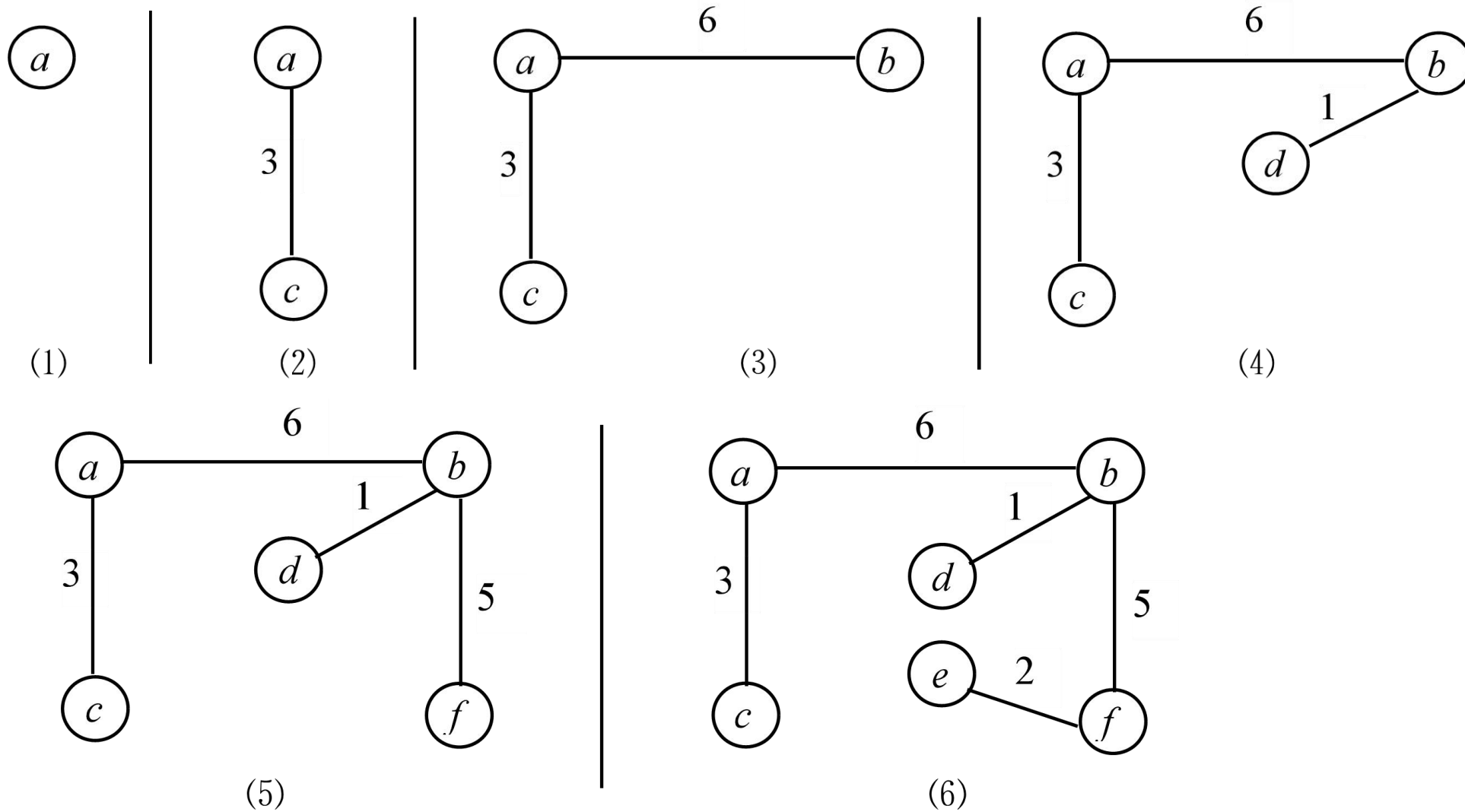
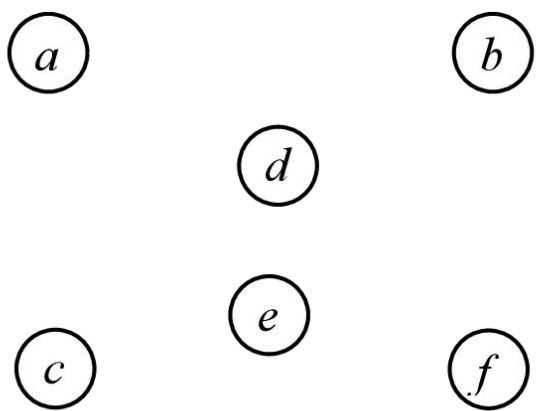


图 6-9 第 (5) 题图

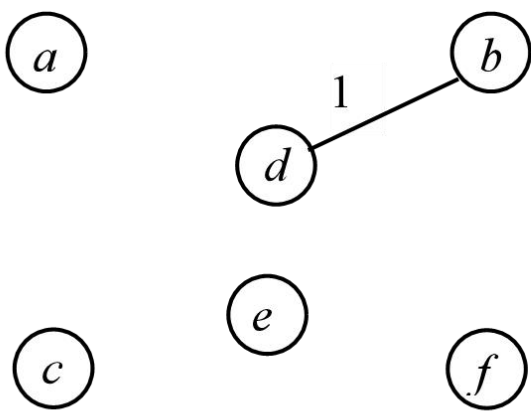
按Prim算法求最小生成树的过程如下：



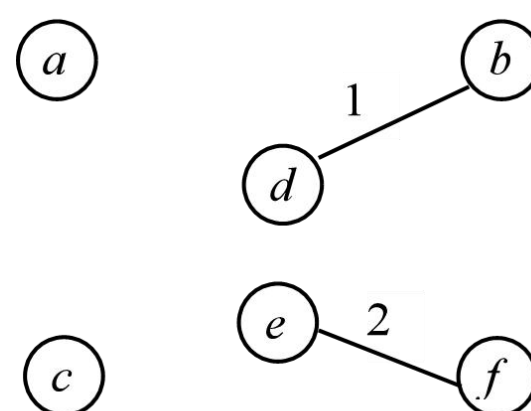
按Kruskal算法求最小生成树的过程如下：



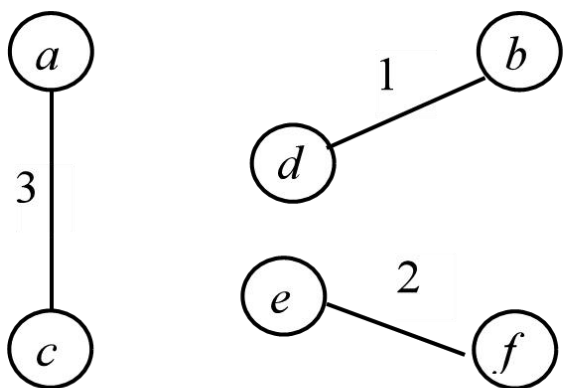
(1)



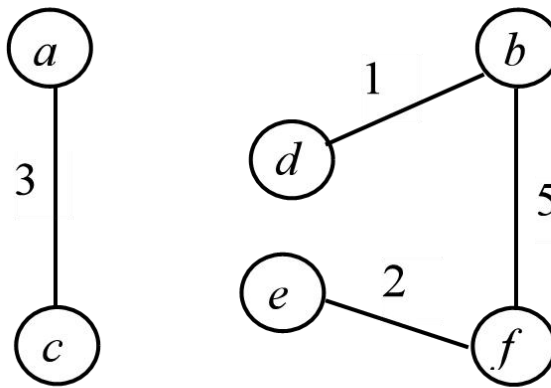
(2)



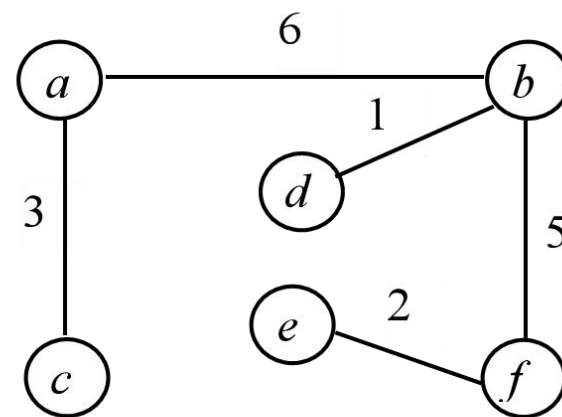
(3)



(4)



(5)



(6)

对如图6-5所示的无向连通网图从顶点d开始用Prim算法构造最小生成树，在构造过程中加入最小生成树的前4条边依次是（ ）。

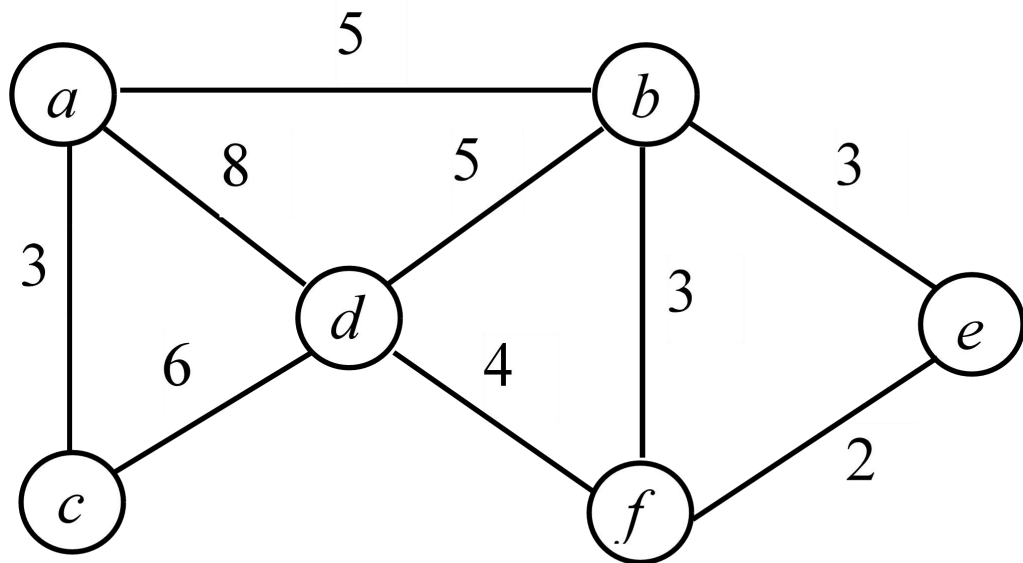


图 6-5 一个无向连通网

先将顶点d涂黑，然后选取一个顶点涂黑另一个顶点未涂黑的权值最小的边，为(d, f)4；然后将顶点f涂黑，选取一个顶点涂黑另一个顶点未涂黑的权值最小的边，为(f, e)2；然后将顶点e涂黑，再选取一个顶点涂黑另一个顶点未涂黑的权值最小的边，为(f, b)3或(e, b)3；然后将顶点b涂黑，再选取一个顶点涂黑另一个顶点未涂黑的权值最小的边，为(b, a)5。

如图6-10所示的有向网图，利用Dijkstra算法求从顶点 v_1 到其他各顶点的最短路径。

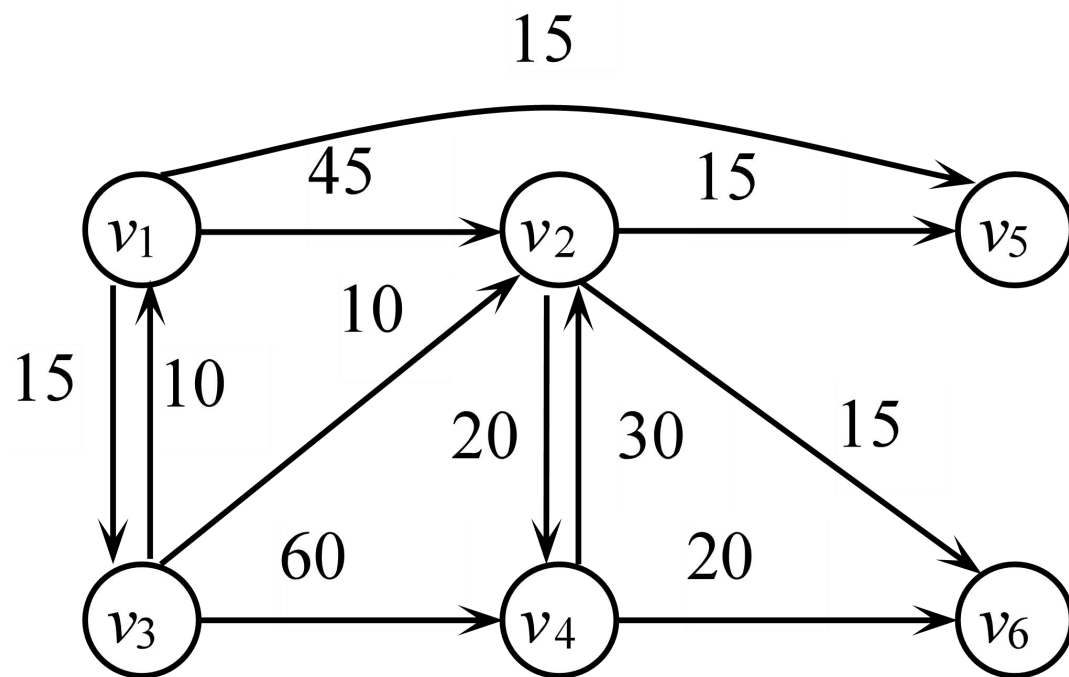


图 6-10 第 (6) 题图

从源点v1到其他各顶点的最短路径如下表所示。

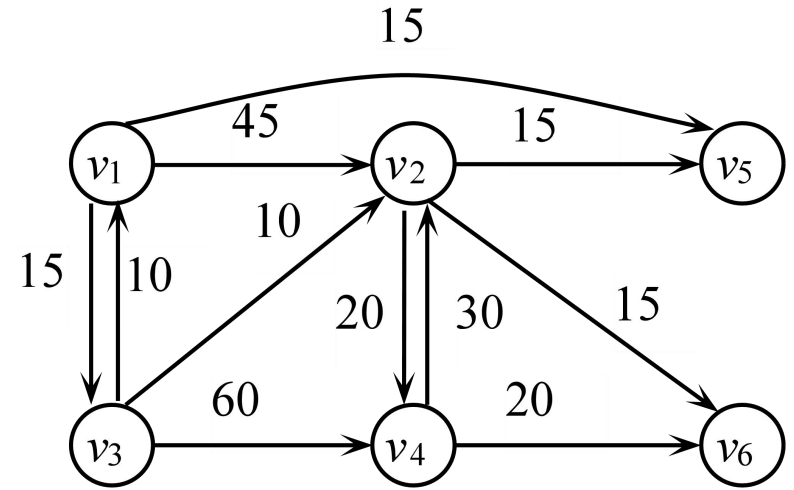


图 6-10 第 (6) 题图

源点	终点	最短路径	最短路径长度
V_1	V_3	$V_1 \quad V_3$	15
V_1	V_5	$V_1 \quad V_5$	15
V_1	V_2	$V_1 \quad V_3 \quad V_2$	25
V_1	V_6	$V_1 \quad V_3 \quad V_2 \quad V_6$	40
V_1	V_4	$V_1 \quad V_3 \quad V_2 \quad V_4$	45

对于如图6-11所示的有向网图，求出各活动的最早开始时间和最晚开始时间，并写出关键活动和关键路径。

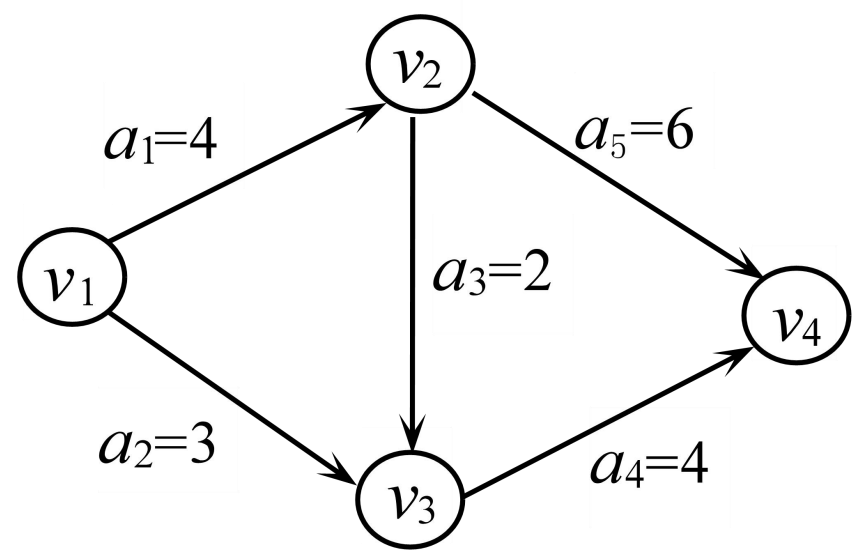


图 6-11 一个有向网图

事件	v_1	v_2	v_3	v_4
最早开始时间	0	4	6	10
最迟开始时间	0	4	6	10

活动	a_1	a_2	a_3	a_4	a_5
最早开始时间	0	0	4	6	4
最迟开始时间	0	3	4	6	4

关键活动是 a_1 、 a_3 、 a_4 和 a_5 ，它们组成两条关键路径，如图6-12所示。

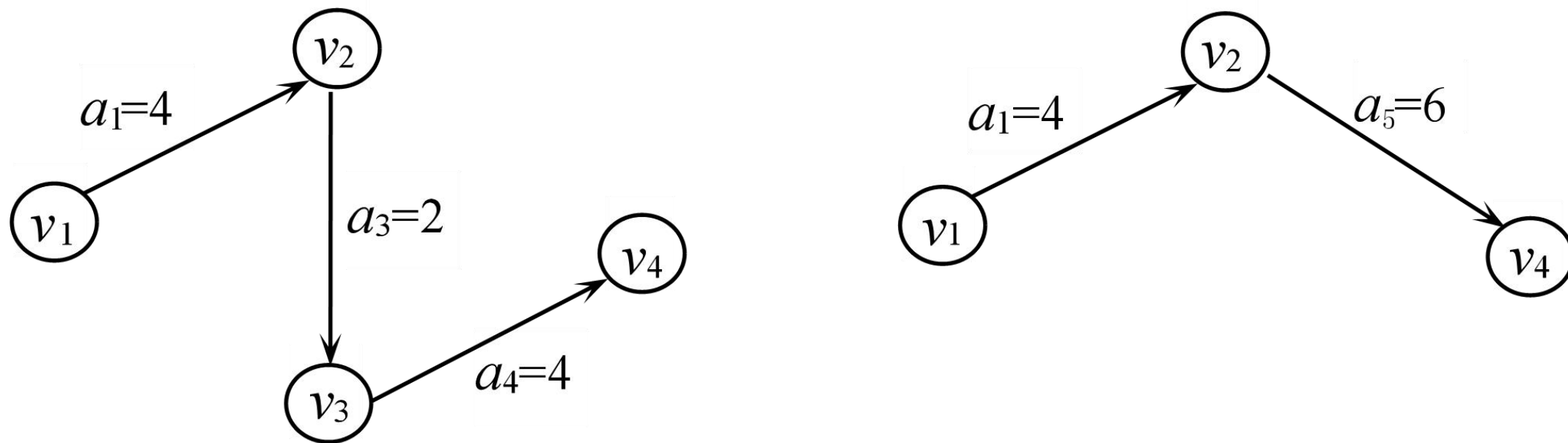


图 6-12 两条关键路径

设有如图6-6所示的AOE网，则事件 v_4 的最早开始时间是（ ），最迟开始时间是（ ），该AOE网的关键路径有（ ）条。

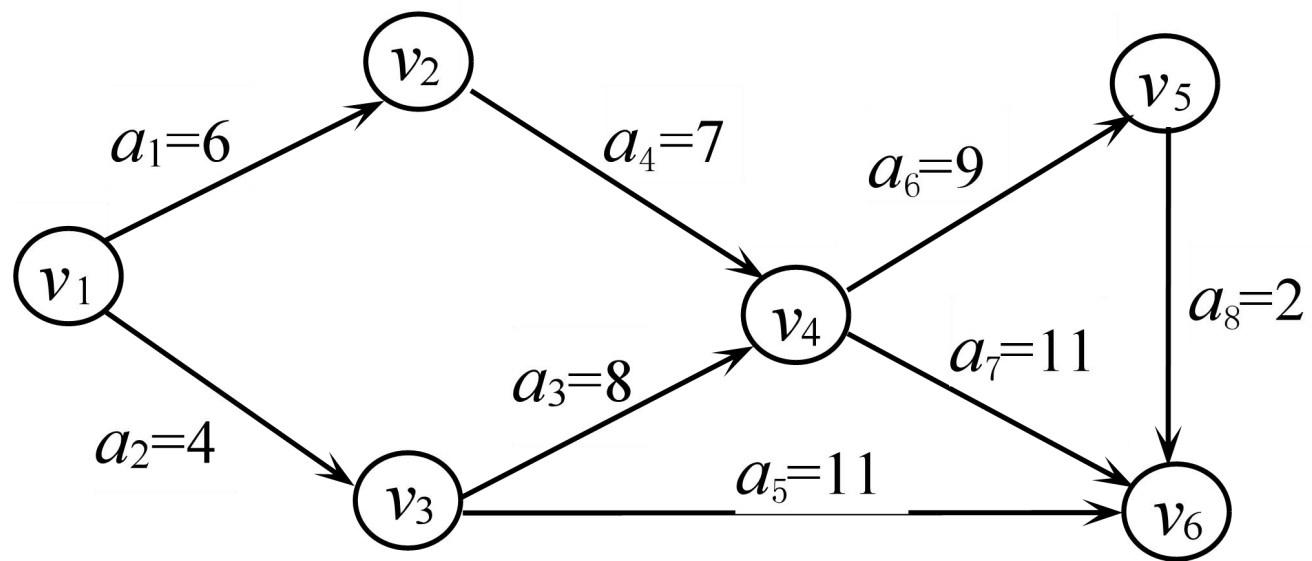
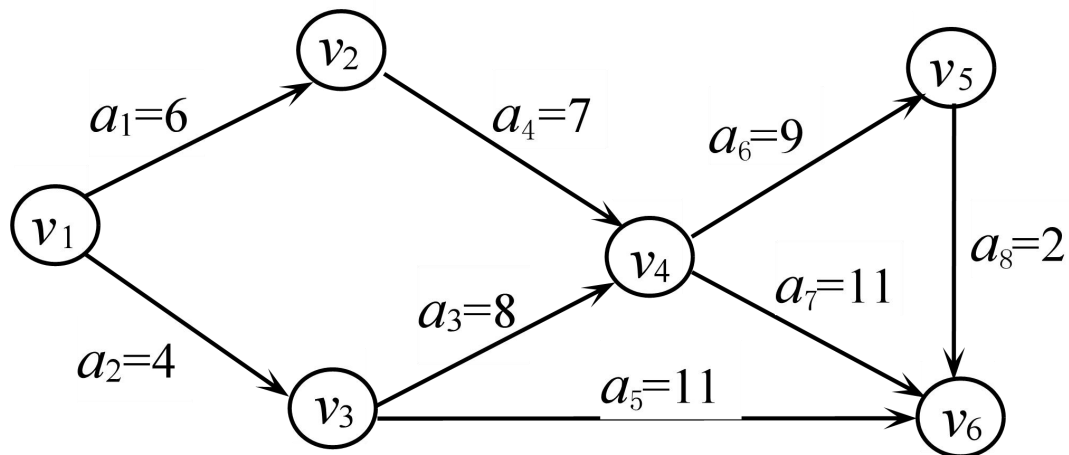


图 6-6 一个 AOE 网



事件 v_4 的最早开始时间是从顶点 v_1 到 v_4 的最长路径长度13，事件 v_4 的最迟开始时间 $=\min\{\text{事件}v_6\text{的最迟发生时间}-11, \text{事件}v_5\text{的最迟发生时间}-9\}=\{24-11, 22-9\}=13$ 。该AOE网的两条关键路径如图6-7所示。

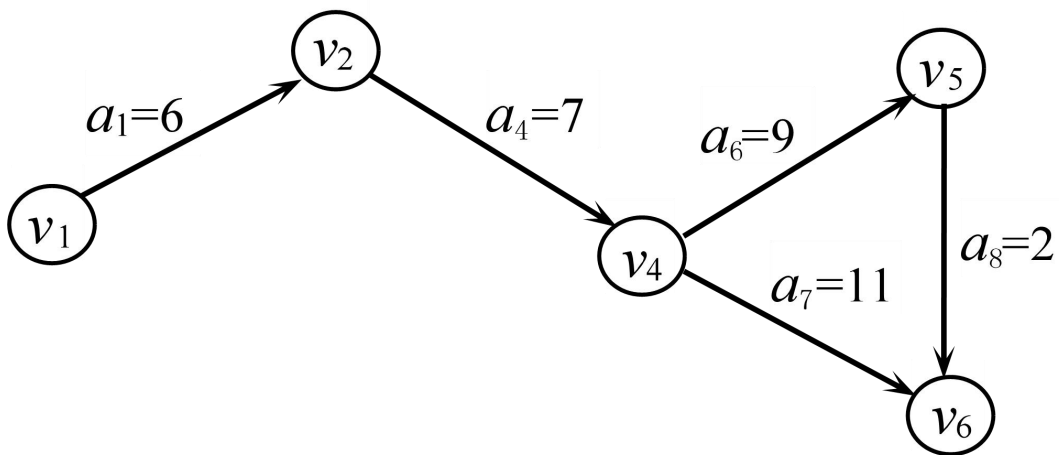


图 6-7 两条关键路径

已知散列函数 $H(k)=k \bmod 12$ ，键值序列为(25, 37, 52, 43, 84, 99, 120, 15, 26, 11, 70, 82)，采用拉链法处理冲突，试构造开散列表，并计算查找成功的平均查找长度。

$H(25)=1$, $H(37)=1$, $H(52)=4$,
 $H(43)=7$, $H(84)=0$, $H(99)=3$,
 $H(120)=0$, $H(15)=3$, $H(26)=2$,
 $H(11)=11$, $H(70)=10$, $H(82)=10$
构造的开散列表如图7-18所示,
平均查找长度
 $ASL=(8\times 1+4\times 2)/12=16/12$ 。

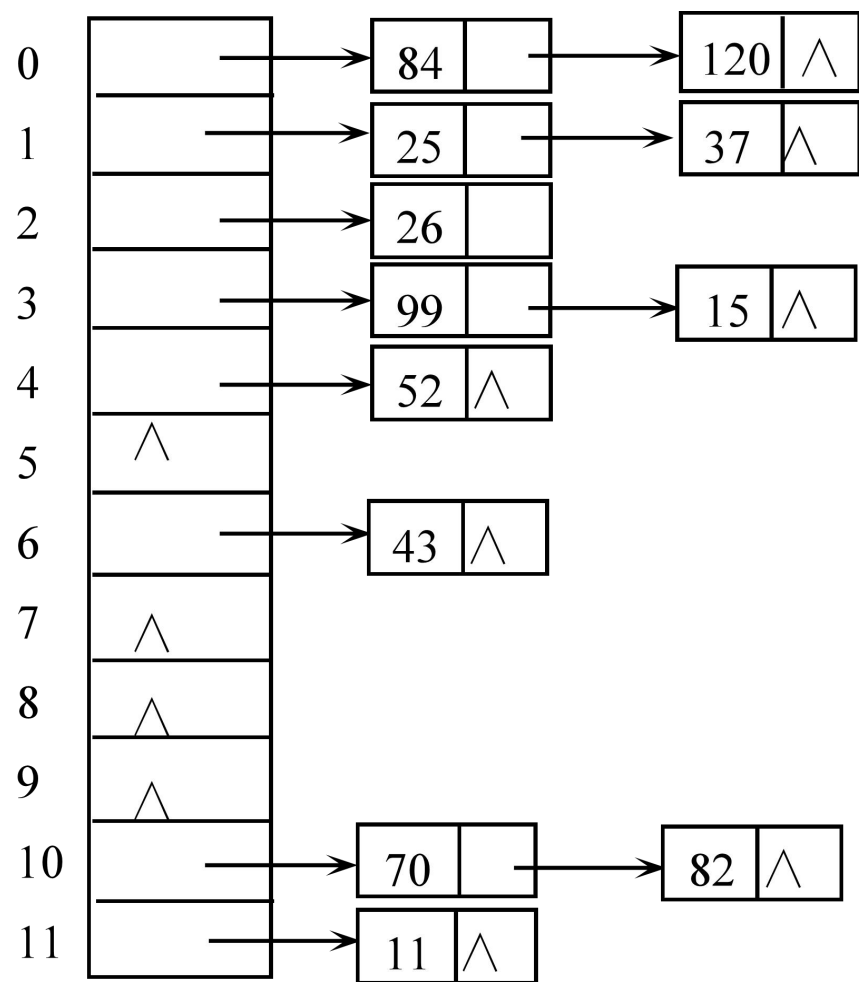
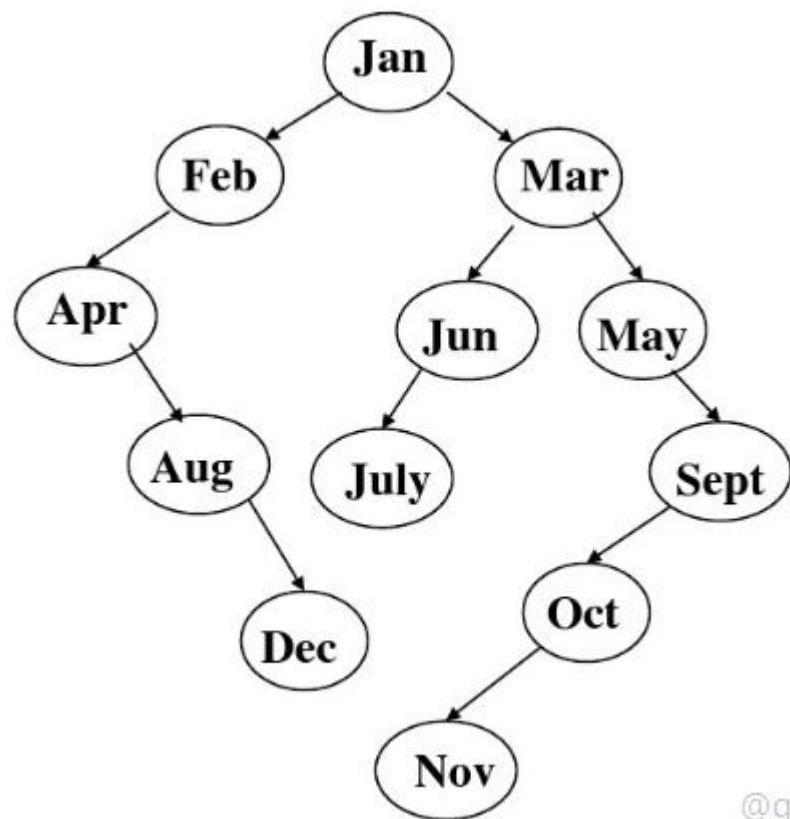
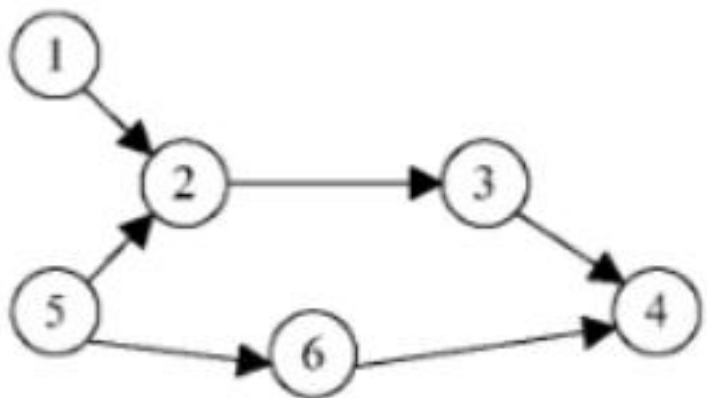


图 7-18 构造的开散列表

已知如下所示长度为12的表 (Jan, Feb, Mar, Apr, May, June, July, Aug, Sep, Oct, Nov, Dec) , 试按表中元素的顺序依次插入一棵初始为空的二叉排序树,画出插入完成之后的二叉排序树



试列出下图中全部可能的拓扑有序序列



152364

152634

156234

512364

512634

516234

561234

判别下列序列是否为堆，如不是，把它调整为堆，用图表示建堆的过程。

① (1 , 5 , 7 , 25 , 21 , 8 , 8 , 42)

② (3 , 9 , 5 , 8 , 4 , 17 , 21 , 6)

序列①是堆，序列②不是堆，调整为堆（假设为大根堆）的过程如图8-6所示。

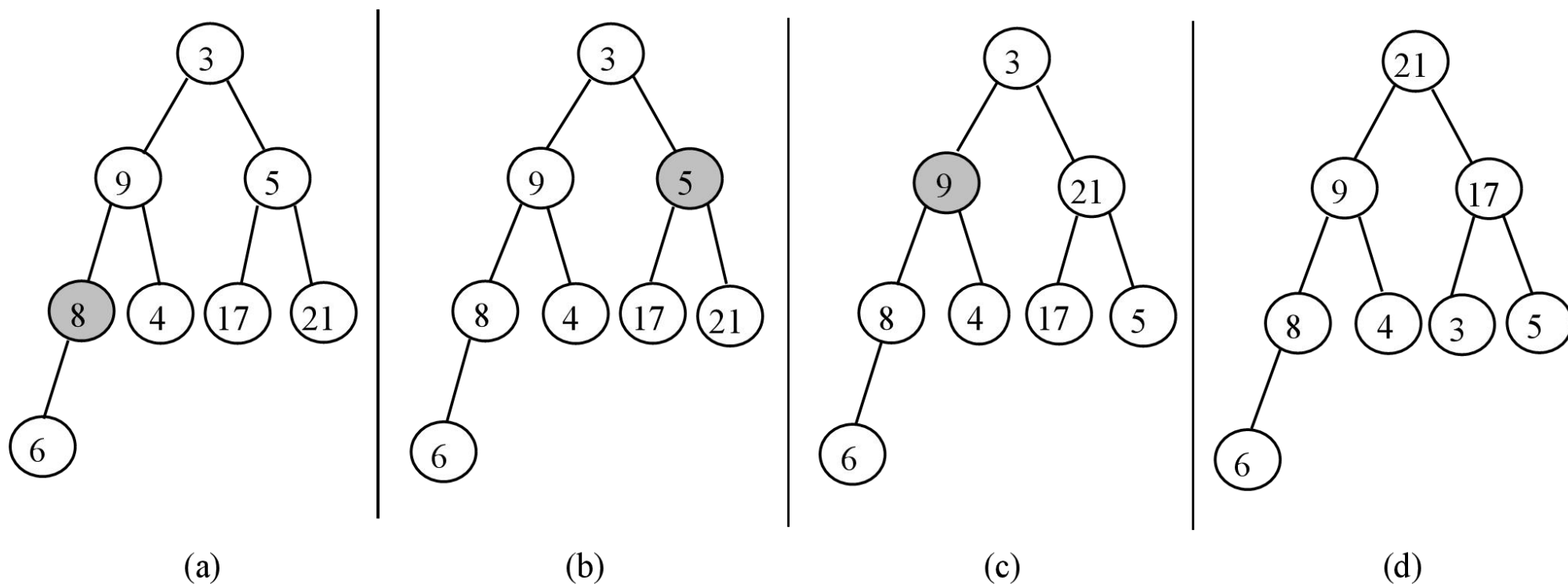


图 8-6 堆调整的过程

已知数据序列为(12, 5, 9, 20, 6, 31, 24), 对该数据序列进行排序, 写出插入排序、起泡排序、快速排序、简单选择排序、堆排序以及二路归并排序每趟的结果。

插入排序：

初始键值序列 [12] 5 9 20 6 31 24

第一趟结果 [5 12] 9 20 6 31 24

第二趟结果 [5 9 12] 20 6 31 24

第三趟结果 [5 9 12 20] 6 31 24

第四趟结果 [5 6 9 12 20] 31 24

第五趟结果 [5 6 9 12 20 31] 24

第六趟结果 [5 6 9 12 20 24 31]

简单选择排序：

初始键值序列 [12 5 9 20 6 31 24]

第一趟结果 5 [12 9 20 6 31 24]

第二趟结果 5 6 [9 20 12 31 24]

第三趟结果 5 6 9 [20 12 31 24]

第四趟结果 5 6 9 12 [20 31 24]

第五趟结果 5 6 9 12 20 [31 24]

第六趟结果 5 6 9 12 20 24 31

快速排序：

初始键值序列 12 5 9 20 6 31 24

第一趟结果 [6 5 9] 12 [20 31 24]

第二趟结果 [5] 6 [9] 12 20 [31 24]

第三趟结果 5 6 9 12 20 [24] 31

第四趟结果 5 6 9 12 20 24 31

起泡排序：

初始键值序列 [12 5 9 20 6 31 24]

第一趟结果 [5 9 12 6 20 24] 31

第二趟结果 [5 9 6] 12 20 24 31

第三趟结果 [5 6] 9 12 20 24 31

第四趟结果 5 6 9 12 20 24 31

堆排序：

初始键值序列 [12 5 9 20 6 31 24]

初始建堆结果 [31 20 24 5 6 9 12]

第一趟结果 [24 20 12 5 6 9] 31

第二趟结果 [20 9 12 5 6] 24 31

第三趟结果 [12 9 6 5] 20 24 31

第四趟结果 [9 5 6] 12 20 24 31

第五趟结果 [6 5] 9 12 20 24 31

第六趟结果 5 6 9 12 20 24 31

二路归并排序：

初始键值序列 12 5 9 20 6 31 24

第一趟结果 [5 12] [9 20] [6 31] [24]

第二趟结果 [5 9 12 20] [6 24 31]

第三趟结果 [5 6 9 12 20 24 31]